

Advanced AI & Software Engineering — Study Notes

These notes are organized as a **technical study guide**, with enough mathematical and implementation detail to support coursework, interviews, research discussions, and building an AI-based educational/agentic system.

1. LangGraph

Definition and Core Idea

LangGraph is a low-level orchestration framework for building **stateful, multi-step, long-running AI agents and workflows as graphs**.

Instead of allowing an LLM to freely decide the entire execution flow, LangGraph lets you explicitly define:

- **State** → information maintained throughout execution
- **Nodes** → operations/functions/agents
- **Edges** → transitions between nodes
- **Conditional edges** → decisions about what happens next
- **Checkpoints** → persistence of execution state
- **Human-in-the-loop** → intervention at selected points

LangGraph is particularly useful when an agent requires **loops, branching, persistence, retries, multiple agents, or controlled planning**. ([Docs by LangChain](#))

Mental Model

Think of it as:

LLM + State Machine + Workflow Engine

rather than simply:

prompt → LLM → answer

Key Components

1. State

The state represents the current knowledge/context of the workflow.

```
from typing import TypedDict
```

```
class AgentState(TypedDict):  
    question: str  
    answer: str  
    confidence: float
```

A node receives state and returns updates.

```
State  
↓  
Node  
↓  
Updated State  
↓  
Next Node
```

LangGraph supports `TypedDict`, dataclasses, and Pydantic models for state schemas. ([Docs by LangChain](#))

2. Nodes

A node is simply a function that performs work.

```
def retrieve(state):  
    ...  
    return {"documents": documents}
```

A node can contain:

- Python logic
- LLM calls
- database operations
- retrieval
- another agent
- API calls
- validation

Importantly, **not every node needs an LLM**.

3. Edges

Edges determine execution flow.

```
START
↓
Retrieve
↓
Generate
↓
Validate
↓
END
```

Conditional edges allow:

```
Generate → Human Review
          |
          |
Generate → Validate
          |
          |
          → Final Answer
```

State Machines

A LangGraph workflow can effectively behave like a **finite-state machine**:

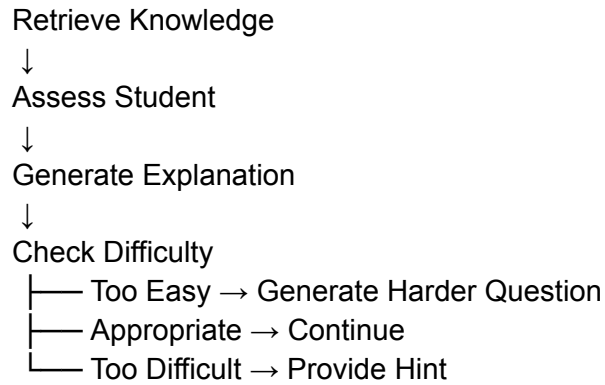
$$S_t \xrightarrow{\text{action}} S_{t+1}$$

where:

- S_t = current state
- action = node execution
- S_{t+1} = updated state

For an educational agent:

```
START
↓
Understand Question
↓
```



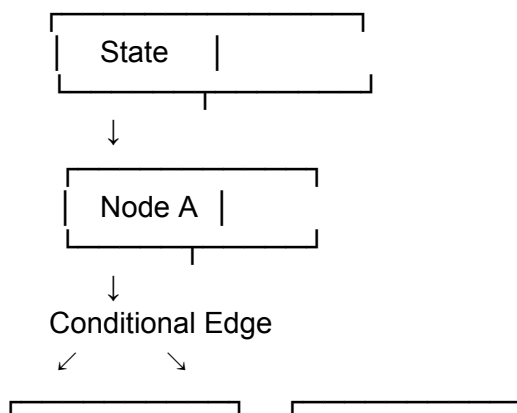
This is much more controllable than asking one LLM prompt to manage the entire workflow.

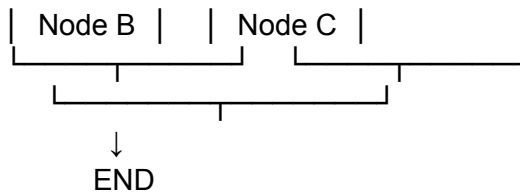
Important Features

- Stateful execution
- Cyclic workflows
- Conditional routing
- Parallel execution
- Persistence/checkpointing
- Human-in-the-loop
- Streaming
- Tool calling
- Multi-agent orchestration
- Durable execution
- Long-running workflows

LangGraph's current architecture emphasizes durable execution, human oversight, memory, streaming, and production deployment. ([Docs by LangChain](#))

Technical Architecture





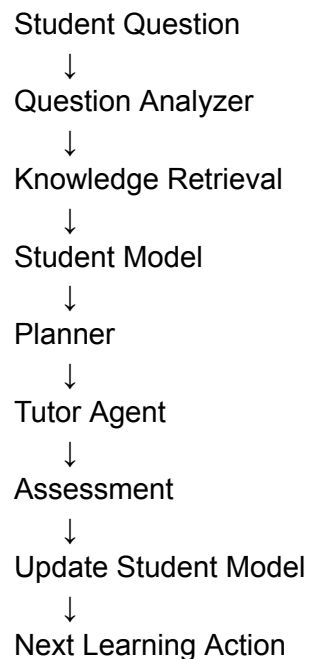
A graph must be compiled before execution. ([LangChain Reference](#))

Applications

- Autonomous agents
- Research agents
- Multi-agent systems
- RAG agents
- Customer-service agents
- Coding agents
- Educational tutors
- Clinical decision-support workflows
- Human-in-the-loop systems
- Approval workflows

Particularly useful for your type of projects

A **student-learning agent** could use:



Resources

- [LangGraph Documentation](#)
 - [LangGraph Graph API](#)
 - [LangGraph Python API Reference](#)
 - [StateGraph Reference](#)
-

2. LangChain

Definition and Core Idea

LangChain is an open-source framework for developing applications and agents powered by LLMs.

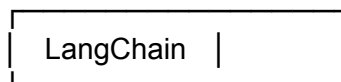
It provides abstractions for:

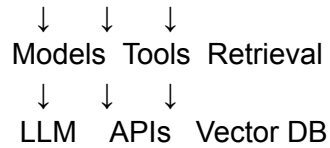
- LLMs
- prompts
- tools
- structured output
- agents
- retrieval
- embeddings
- document processing
- integrations

The current LangChain ecosystem provides standardized model/tool interfaces and higher-level agent abstractions. ([Docs by LangChain](#))

Core Architecture

Application





Major Components

Models

Connect to:

- OpenAI
- Anthropic
- Google
- Hugging Face
- Ollama
- other providers

Prompts

Templates for constructing model inputs.

Tools

Functions that an agent can invoke.

```
def calculate(x, y):  
    return x + y
```

The LLM can decide when to invoke the function.

Retrievers

Retrieve relevant information from:

- vector databases
- documents
- APIs
- search engines

Agents

An agent combines:

Agent=Model+Instructions+Tools+Execution Loop
Agent = Model + Instructions + Tools + Execution\ Loop

LangChain vs LangGraph

Feature	LangChain	LangGraph
Abstraction	Higher-level	Lower-level
Agents	Yes	Yes
Explicit graph	Not necessary	Core concept
State machine	Limited/indirect	Central
Complex branching	Less suitable	Excellent
Persistence	Supported ecosystem	Core orchestration capability
Human-in-loop	Supported	Strong support
Complex workflows	Good	Excellent
Beginner-friendly	Higher	Lower

LangChain's own documentation positions LangChain as the higher-level agent framework and LangGraph as the lower-level orchestration/runtime layer. ([Docs by LangChain](#))

Simple rule

Use LangChain when you want to build an LLM application quickly.

Use LangGraph when you need precise control over agent execution.

They are complementary rather than competing technologies.

Resources

- [LangChain Documentation](#)
 - [LangChain API Reference](#)
-

3. Knowledge Tracing

Definition

Knowledge Tracing (KT) is the task of modeling how a student's knowledge changes over time based on their interactions with learning activities.

The fundamental objective is:

$$P(K_t | O_{1:t})$$

where:

- K_t = student's knowledge state at time t
- $O_{1:t}$ = observed interactions up to time t

Typical observations:

- correct answer
 - incorrect answer
 - response time
 - hint usage
 - number of attempts
 - question difficulty
-

Core Idea

Suppose a student answers:

Q1: Correct

Q2: Correct

Q3: Incorrect

Q4: Correct

The system should infer something like:

Probability student knows Algebra = 0.82

Probability student knows Fractions = 0.61

Probability student knows Geometry = 0.35

The system then uses this information to decide what the student should study next.

Types of Knowledge Tracing

3.1 Bayesian Knowledge Tracing — BKT

BKT models knowledge as a **latent binary variable**.

For each skill:

$$K_t \in \{0, 1\} \quad K_t \in \{0, 1\}$$

where:

- 0 = not mastered
- 1 = mastered

Classic BKT uses four parameters:

Initial knowledge

$$P(L_0)P(L_0)$$

Probability the student initially knows the skill.

Learning probability

$$P(T)P(T)$$

Probability of learning between opportunities.

Guess

$$P(G)P(G)$$

Probability of answering correctly despite not knowing.

Slip

$$P(S)P(S)$$

Probability of answering incorrectly despite knowing.

BKT Update

If the student answers correctly:

$$P(L_t | \text{correct}) = \frac{P(L_t)(1-S)}{P(L_t)(1-S) + (1-P(L_t))G}$$

Then learning transition:

$$P(L_{t+1}) = P(L_t | \text{correct}) + (1 - P(L_t | \text{correct}))T$$

BKT is interpretable and relatively simple but makes strong assumptions about learning dynamics.

3.2 Deep Knowledge Tracing — DKT

DKT uses neural networks, particularly **RNNs/LSTMs**, to model student interactions.

The input sequence:

$$x_1, x_2, \dots, x_{t-1}, x_t$$

is processed by an RNN:

$$h_t = f(x_t, h_{t-1})$$

Then:

$$\hat{y}_{t+1} = \text{softmax}(W h_t + b)$$

The hidden state h_t acts as a learned representation of student knowledge.

The original DKT work demonstrated that RNNs can model student learning without explicitly encoding domain knowledge. ([arXiv](#))

3.3 Other Important KT Families

Dynamic Bayesian Networks

Extend BKT's probabilistic structure.

DKVMN

Dynamic Key-Value Memory Networks

Separates:

- knowledge concepts → keys
- student's mastery → values

SAKT

Self-Attentive Knowledge Tracing

Uses attention/Transformer-style mechanisms.

AKT

Attentive Knowledge Tracing

Uses attention and difficulty-aware modeling.

Graph-based KT

Models relationships between concepts using graphs.

LLM-based KT

Uses LLMs to interpret richer student interactions:

Question

Answer

Explanation

Reasoning

Hint requests

Free-text response

Applications

- Personalized learning
- Intelligent tutoring systems
- Adaptive quizzes
- Curriculum recommendation
- Early identification of struggling students
- Skill mastery estimation

- Question recommendation
 - Learning analytics
-

Resources

- [Deep Knowledge Tracing paper](#)
- [A Survey of Knowledge Tracing](#)

The survey is particularly useful because it organizes KT into major model families and discusses variants and applications. ([arXiv](#))

4. Item Response Theory — IRT

Definition

Item Response Theory (IRT) is a family of psychometric models describing the probability that a person answers an assessment item correctly as a function of:

1. the person's latent ability
2. characteristics of the item

Let:

$\theta = \text{student ability}$

Then:

$P(X=1 | \theta)$

represents the probability of answering an item correctly.

1PL / Rasch Model

The simplest model:

$P(X=1 | \theta) = \frac{1}{1 + e^{-(\theta - b)}}$

where:

- θ = student ability
 - b = item difficulty
-

2PL Model

Adds discrimination:

$$P(X=1 | \theta) = \frac{1}{1 + e^{-a(\theta - b)}} \quad P(X=1 | \theta) = \frac{1}{1 + e^{-a(\theta - b)}}$$

where:

- a = discrimination
- b = difficulty

High a :

Small changes in ability produce large changes in probability of success.

3PL Model

Adds guessing:

$$P(X=1 | \theta) = c + (1 - c) \frac{1}{1 + e^{-a(\theta - b)}} \quad P(X=1 | \theta) = c + (1 - c) \frac{1}{1 + e^{-a(\theta - b)}}$$

where:

- c = guessing parameter
-

Interpretation

Imagine:

Student Ability: $\theta = 0.8$

Question A:

difficulty $b = -1$

→ easy

Question B:

difficulty $b = 0.8$

→ appropriately challenging

Question C:

difficulty $b = 2$

→ difficult

This allows adaptive testing systems to select questions appropriate to the student's estimated ability.

Item Information

IRT also estimates how informative an item is.

For a 2PL model:

$$I(\theta) = a^2 P(\theta) [1 - P(\theta)]$$

The system can choose an item maximizing:

$$\operatorname{argmax}_i I_i(\theta)$$

This is one of the foundations of **Computerized Adaptive Testing (CAT)**.

IRT vs Knowledge Tracing

Feature	IRT	Knowledge Tracing
Primary focus	Ability/item characteristics	Knowledge evolution
Time	Usually static	Explicitly sequential
Student state	Latent ability	Changing knowledge
Item difficulty	Central	Can be included

Adaptive testing	Excellent	Useful
Learning process	Limited	Central

Combining them

A powerful educational system can combine:

IRT
↓
Estimate question difficulty
↓
Knowledge Tracing
↓
Estimate student's current mastery
↓
Adaptive Planner
↓
Select next question

Resources

- [Lord's Applications of Item Response Theory](#)
- [ETS reference to Lord's IRT book](#)

Lord's 1980 work remains a foundational reference for practical applications of IRT, including item parameters, ability estimates, and information functions. ([ERIC](#))

5. Pydantic v2

Definition

Pydantic is a Python library for defining structured data models and performing validation, parsing, and serialization using Python type annotations.

Example:

```
from pydantic import BaseModel
```



```
class Student(BaseModel):  
    name: str  
    age: int  
    mastery: float
```

Input:

```
Student(  
    name="Ali",  
    age=22,  
    mastery=0.83  
)
```

Pydantic validates the data against the declared schema.

Why It Matters in AI Systems

LLMs produce probabilistic outputs.

Your application needs deterministic structure.

LLM

↓

Unstructured / uncertain output

↓

Pydantic validation

↓

Reliable application object

For example:

```
class TutorDecision(BaseModel):  
    concept: str  
    difficulty: int  
    explanation: str  
    confidence: float
```

Important Pydantic v2 Changes

Pydantic v2 introduced significant API changes and a new validation architecture. ([Pydantic](#))

Pydantic v1	Pydantic v2
<code>dict()</code>	<code>model_dump()</code>
<code>json()</code>	<code>model_dump_json()</code>
<code>parse_obj()</code>	<code>model_validate()</code>
<code>copy()</code>	<code>model_copy()</code>
<code>construct()</code>	<code>model_construct()</code>
<code>json_schema()</code>	<code>model_json_schema()</code>
<code>__fields__</code>	<code>model_fields</code>

Example:

v1

```
student.dict()
```

v2

```
student.model_dump()
```

Validation

```
Student.model_validate(data)
```

Important Features

- Type validation
- Nested models
- Custom validators
- Serialization
- JSON schema generation
- Strict validation

- Computed fields
- Generic models
- Dataclasses
- Settings management

Custom validation

from pydantic import BaseModel, field_validator

```
class Student(BaseModel):
    mastery: float

    @field_validator("mastery")
    @classmethod
    def valid_mastery(cls, value):
        if not 0 <= value <= 1:
            raise ValueError("Mastery must be between 0 and 1")
        return value
```

Resources

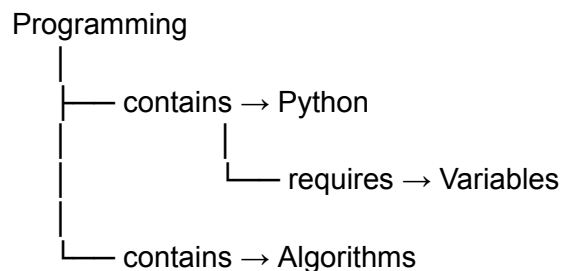
- [Pydantic v2 Migration Guide](#)
 - [Pydantic Documentation](#)
-

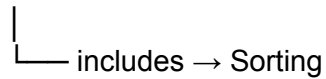
6. Concept Graphs

Definition

A **concept graph** represents knowledge as interconnected concepts.

Example:





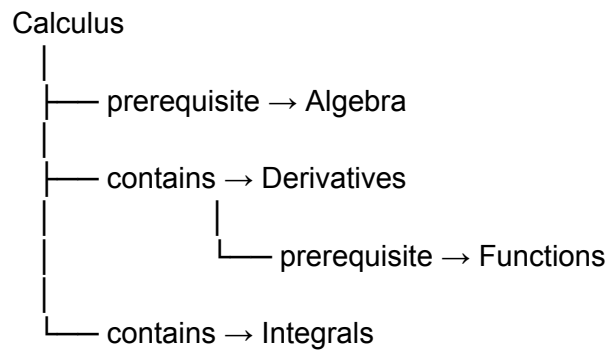
Mathematically:

$G=(V,E)$

where:

- V = concepts/entities
- E = relationships

Example Educational Concept Graph



This graph can be used by a tutoring system to understand prerequisites.

NetworkX

NetworkX is a Python library for creating, manipulating, and analyzing graphs. ([NetworkX](#))

Example:

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
G.add_edge(  
    "Derivatives",  
    "Calculus",
```

```
    relation="part_of"  
  )
```

You can calculate:

- degree centrality
 - shortest paths
 - PageRank
 - connected components
 - community structures
 - graph traversal
-

Neo4j

Neo4j is a graph database using a **property graph model**.

It stores:

Nodes

Relationships

Properties

For example:

```
(Student)-[:KNOWS]->(Concept)
```

Neo4j relationships have direction/type and can carry properties. ([Neo4j Graph Intelligence Platform](#))

Cypher

Neo4j's query language is **Cypher**.

Example:

```
MATCH (s:Student)-[:MASTERED]->(c:Concept)  
RETURN s, c
```

Prerequisite query:

```
MATCH (c:Concept)-[:PREREQUISITE]->(p:Concept)
RETURN c, p
```

NetworkX vs Neo4j

Feature	NetworkX	Neo4j
Type	Python library	Graph database
Persistence	Usually application-managed	Native
Large production graphs	Limited	Strong
Algorithms	Excellent	Graph Data Science
Visualization	Good	Good
Query language	Python	Cypher
Best for	Analysis/prototyping	Persistent graph applications

Rule

Use **NetworkX** when the graph primarily lives in your Python process.

Use **Neo4j** when the graph itself is an important persistent data asset.

Resources

- [NetworkX Documentation](#)
 - [Neo4j Graph Database Documentation](#)
 - [Neo4j Getting Started](#)
-

7. Knowledge Base / RAG

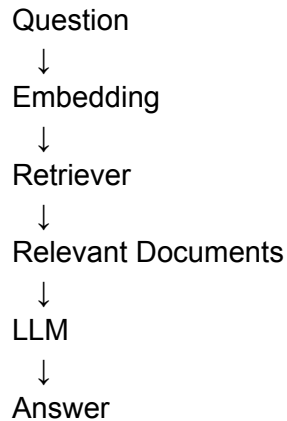
Definition

Retrieval-Augmented Generation (RAG) combines information retrieval with LLM generation.

Instead of:

Question → LLM → Answer

RAG uses:



The original RAG formulation was introduced for knowledge-intensive NLP tasks by Lewis et al. (nlp.cs.ucl.ac.uk)

RAG Pipeline

Step 1 — Document ingestion

PDF
Web page
Database
Notes
↓
Document loader

Step 2 — Chunking

Large documents are divided:

Document

↓
Chunk 1
Chunk 2
Chunk 3
...

Step 3 — Embedding

A neural encoder converts text into vectors:

$$f(\text{text}) = \mathbf{v} \in \mathbb{R}^d \quad f(\text{text}) = \mathbf{v} \in \mathbb{R}^d$$

Step 4 — Storage

Store:

vector
+
text
+
metadata

Step 5 — Retrieval

Given query q :

$$\mathbf{v}_q = f(q)$$

Find:

$$\text{TopK}(\mathbf{v}_q, V)$$

using similarity.

Cosine similarity

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \quad \text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Step 6 — Generation

Question
+
Retrieved Context
↓

LLM



Grounded Answer

Chroma

Chroma is an open-source AI data infrastructure/vector database supporting embeddings, metadata, dense/sparse/hybrid retrieval, filtering, and multimodal retrieval. ([Chroma Docs](#))

Good for:

- prototypes
 - local RAG
 - small/medium applications
 - experimentation
 - quick development
-

Qdrant

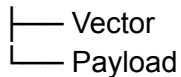
Qdrant is a vector search engine/database designed for semantic search and AI applications.

Its data model includes:

Collection



Points



It supports dense, sparse, and multivector representations and uses HNSW for dense vector indexing. ([Qdrant](#))

Chroma vs Qdrant

Feature	Chroma	Qdrant
---------	--------	--------

Beginner friendly	★★★★★	★★★★★
Local development	Excellent	Excellent
Production scale	Good	Excellent
Metadata filtering	Yes	Strong
Dense vectors	Yes	Yes
Sparse vectors	Yes	Yes
Hybrid search	Yes	Strong
HNSW	Supported internally	Core dense index
Multivector	Less central	Strong
Cloud	Yes	Yes
Best use	RAG prototypes/apps	Production retrieval

Practical recommendation

For learning:

Start with Chroma.

For a serious production/research retrieval system:

Learn Qdrant as well.

Resources

- [RAG paper — Lewis et al.](#)
 - [Chroma Documentation](#)
 - [Qdrant Documentation](#)
 - [Qdrant Vector Search Guide](#)
-

8. Student Model Storage

A student model should maintain persistent information such as:

Student

- |— profile
 - |— concepts
 - |— mastery probabilities
 - |— assessment history
 - |— interaction history
 - |— learning events
 - |— recommendations
-

SQLite

SQLite is an **embedded, serverless, zero-configuration relational database**. It stores the database directly in files rather than requiring a database server. ([SQLite](#))

Example:

student.db

Advantages:

- extremely simple
- no server
- portable
- excellent for local applications
- excellent for prototypes

Disadvantages:

- less suitable for highly concurrent production systems
 - limited distributed architecture
-

PostgreSQL

PostgreSQL is a full client-server relational database designed for robust production workloads.

Useful features include:

- transactions

- foreign keys
- indexes
- advanced SQL
- JSON/JSONB
- concurrency control
- extensions
- replication ecosystem

The current PostgreSQL documentation covers SQL, transactions, indexes, concurrency, advanced features, and administration. ([PostgreSQL](#))

SQLAlchemy

SQLAlchemy is a Python SQL toolkit and ORM.

Instead of writing:

```
SELECT * FROM students WHERE id = 1;
```

you can work with Python objects:

```
student = session.get(Student, 1)
```

SQLAlchemy 2.x provides both:

- Core
- ORM

and its current documentation recommends the Unified Tutorial for learning modern SQLAlchemy. ([SQLAlchemy Documentation](#))

SQLite vs PostgreSQL

Feature	SQLite	PostgreSQL
Architecture	Embedded	Client-server
Setup	Almost zero	Requires service

Local prototype	★★★★★	★★★★
Production backend	Limited	★★★★★
Concurrency	Limited	Excellent
Scaling	Limited	Strong
Complex queries	Good	Excellent
Distributed deployment	No	Yes
Best use	Prototype/local app	Production

Recommended architecture

Development:

```
FastAPI
↓
SQLAlchemy
↓
SQLite
```

Production:

```
FastAPI
↓
SQLAlchemy
↓
PostgreSQL
```

9. FastAPI

Definition

FastAPI is a modern Python web framework for building APIs using Python type annotations.

It integrates particularly well with:

- Pydantic
- async/await

- OpenAPI
 - Swagger UI
 - SQLAlchemy
 - authentication
 - AI/ML services
-

Basic Architecture

Frontend
↓
HTTP Request
↓
FastAPI
↓
Pydantic Validation
↓
Business Logic
↓
Database / AI Agent
↓
JSON Response

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Question(BaseModel):
    text: str

@app.post("/ask")
async def ask_question(question: Question):
    return {"answer": "Example"}
```

Async Support

FastAPI supports:

```
async def  
await
```

For example:

```
@app.get("/student/{id}")  
async def get_student(id: int):  
    result = await database_call()  
    return result
```

This is particularly useful for I/O-bound operations such as:

- database queries
- API calls
- LLM calls
- retrieval
- network requests

FastAPI supports mixing `def` and `async def` depending on whether underlying operations are awaitable. ([FastAPI](#))

Pydantic Integration

FastAPI automatically uses Pydantic models for request validation.

```
class Student(BaseModel):  
    name: str  
    mastery: float
```

Then:

```
@app.post("/students")  
async def create_student(student: Student):  
    ...
```

FastAPI validates incoming JSON automatically.

Pydantic models can also be used for forms and response schemas. ([FastAPI](#))

Applications

- AI APIs
 - RAG systems
 - ML inference APIs
 - agent backends
 - authentication services
 - educational platforms
 - microservices
 - model-serving systems
-

Resources

- [FastAPI Async Documentation](#)
 - [FastAPI Documentation](#)
 - [Pydantic Documentation](#)
-

10. Agent Planning with LangGraph

Definition

Agent planning is the process through which an AI system determines:

What actions should I perform, in what order, to achieve a goal?

A basic LLM agent might do:

Goal

↓

LLM

↓

Tool

↓

LLM

↓

Tool

↓

Answer

A structured planner instead explicitly represents execution.

Planning as a State Machine

Consider:

"Teach this student derivatives."

The planner could create:

START



Assess Student



Retrieve Concept Graph



Identify Missing Prerequisites



Create Learning Plan



Teach Concept



Generate Exercise



Evaluate Answer



Update Knowledge State



Mastered?



— NO → Remediate



— YES → Next Concept

This is essentially a **closed-loop control system**.

Closed-Loop Agent

Instead of:

Goal→Plan→ExecuteGoal \rightarrow Plan \rightarrow Execute

use:

Goal→Plan→Execute→Observe→Evaluate→Update→ReplanGoal \rightarrow Plan \rightarrow Execute \rightarrow Observe \rightarrow Evaluate \rightarrow Update \rightarrow Replan

This is much more robust.

Example State

```
class StudentState(TypedDict):
    student_id: int
    question: str
    concepts: list[str]
    mastery: dict[str, float]
    retrieved_context: list[str]
    plan: list[str]
    current_step: int
    answer: str
    confidence: float
```

Possible nodes:

```
analyze_question
retrieve_knowledge
retrieve_student_model
plan_lesson
teach
generate_question
evaluate
update_student_model
```

Conditional routing:

```
if mastery < 0.5:
    return "remediate"
```

```
elif mastery < 0.8:
    return "practice"
```

```
else:  
    return "advance"
```

Agent Planning Strategies

1. Fixed Workflow

$A \rightarrow B \rightarrow C \rightarrow D$

Best when the process is predictable.

2. Conditional Workflow

A
↓
B
├── condition 1 → C
└── condition 2 → D

Best for decision systems.

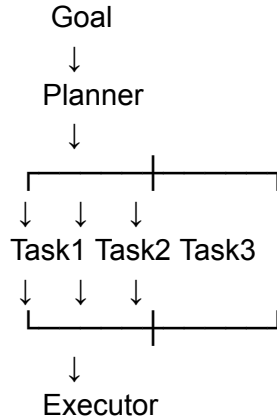
3. ReAct

Conceptually:

Reason
↓
Act
↓
Observe
↓
Reason
↓
Act

Useful for tool-using agents.

4. Planner–Executor



5. Hierarchical Planning

High-level Goal



Subgoal



Tasks



Actions

Useful for complicated autonomous systems.

Why LangGraph Is Useful for Planning

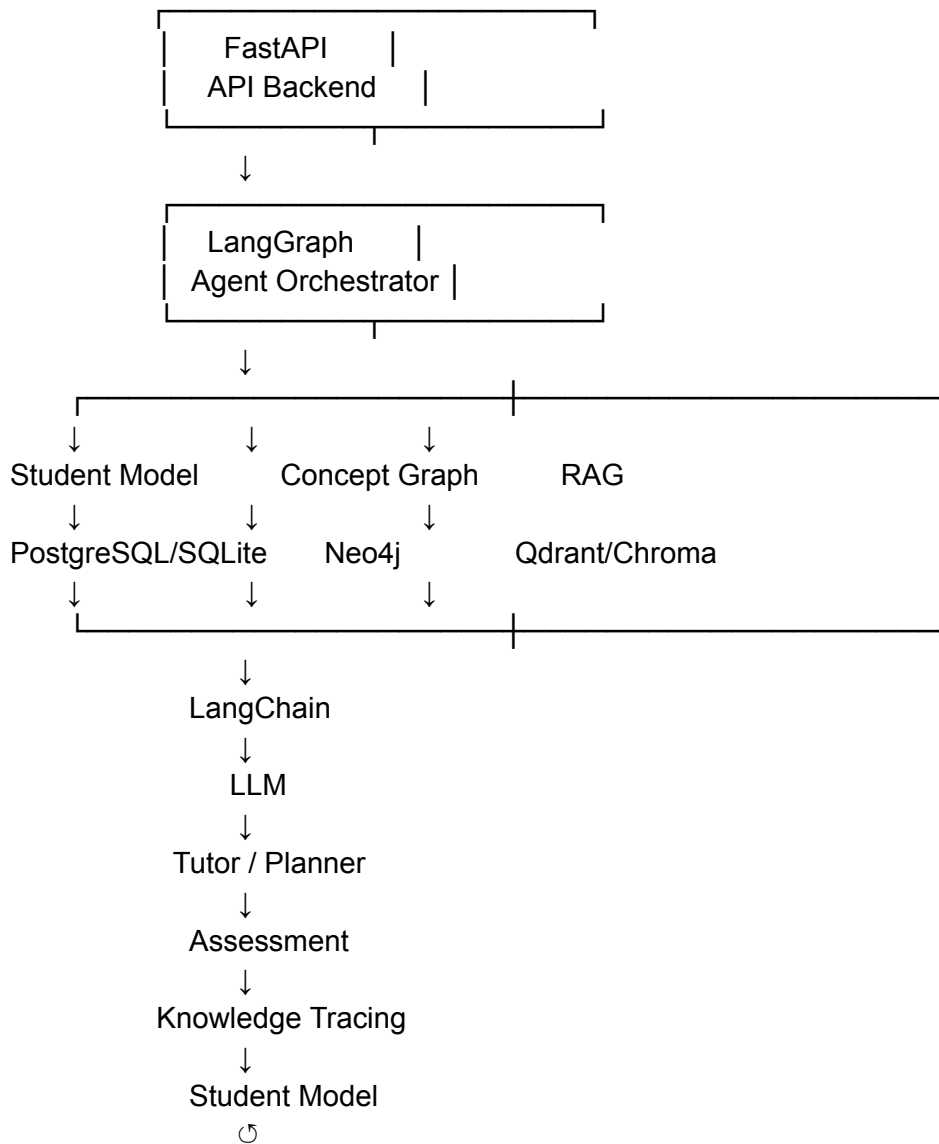
LangGraph's graph abstraction allows you to explicitly represent:

- planning
- execution
- evaluation
- retry
- branching
- loops
- human intervention
- persistent state

The graph therefore becomes the **control structure of the agent**, while the LLM provides intelligence inside selected nodes. ([Docs by LangChain](#))

Integrated Architecture

For an advanced **AI-powered adaptive learning system**, all these technologies can fit together:



This architecture combines **symbolic structure + probabilistic student modeling + retrieval + LLM reasoning + deterministic orchestration**.

How the Concepts Relate

The most important thing to understand is that these are **not isolated technologies**.

LangChain

Provides the **LLM application/agent abstractions**.

↓

LangGraph

Controls the **workflow and agent state**.

↓

Pydantic

Ensures **structured, validated data**.

↓

FastAPI

Exposes the system through **HTTP APIs**.

↓

SQLAlchemy + PostgreSQL

Stores the **persistent student/application state**.

↓

Concept Graph

Represents **relationships between concepts**.

↓

Knowledge Tracing

Estimates **what the student knows**.

↓

IRT

Models **student ability and item characteristics**.

↓

RAG

Provides the agent with **external/domain knowledge**.

↓

Chroma/Qdrant

Provide the **semantic retrieval infrastructure**.

High-Level Comparison Cheat Sheet

Concept	Main Question It Answers
LangChain	How do I build an LLM application/agent?
LangGraph	How should my agent execute and transition between states?
Knowledge Tracing	What does the student know right now?
BKT	How does mastery probability change after an interaction?
DKT	Can a neural network learn the student's knowledge state?
IRT	How does student ability relate to item difficulty?
Pydantic	Is my data structured and valid?
Concept Graph	How are concepts related?
RAG	What external information should the LLM retrieve?
Chroma	How can I store/retrieve embeddings easily?

Qdrant	How can I perform scalable semantic/hybrid retrieval?
SQLite	How can I persist data locally with minimal infrastructure?
PostgreSQL	How can I persist application data reliably at production scale?
SQLAlchemy	How can Python applications interact cleanly with SQL databases?
FastAPI	How can I expose my AI system as an API?
Agent Planning	What actions should the AI perform and in what order?

Recommended Learning Order

If your goal is to **actually build an advanced AI educational/agentic system**, don't learn these randomly.

Phase 1 — Software Foundations

1. Python typing
2. Pydantic v2
3. SQL
4. SQLite
5. PostgreSQL
6. SQLAlchemy
7. FastAPI

Phase 2 — LLM Engineering

8. LangChain
9. Embeddings
10. Vector databases
11. RAG
12. Chroma
13. Qdrant

Phase 3 — Agent Engineering

14. LangGraph fundamentals
15. State
16. Nodes

- 17. Edges
- 18. Conditional routing
- 19. Persistence/checkpointing
- 20. Human-in-the-loop
- 21. Agent planning
- 22. Multi-agent workflows

Phase 4 — Educational AI

- 23. Concept graphs
- 24. Knowledge Tracing
- 25. BKT
- 26. DKT
- 27. IRT
- 28. Adaptive testing
- 29. Personalized learning

Phase 5 — Integration

Build:

FastAPI
+
PostgreSQL
+
SQLAlchemy
+
LangGraph
+
LangChain
+
Qdrant
+
Neo4j
+
BKT/DKT
+
IRT
+
LLM

That combination would give you a genuinely strong foundation in **agentic AI + educational AI + backend engineering + knowledge representation**, rather than simply learning how to call an LLM API.

